V1.0 — OUT 2026

GUIA DE REFERÊNCIA TÉCNICA

Modelos de Áudio IA Locais para Tauri + Leptos

Decisões finais de stack, arquitetura de integração e passos de instalação detalhados para ASR e TTS em PT-BR rodando 100% on-device, com runtime Rust (sherpa-onnx) linkado estaticamente ao backend Tauri 2.

Rust · Tauri 2.12 · Leptos · sherpa-onnx 2.0.1 · Whisper ONNX · Kokoro v1.0 · Piper PT-BR



STACK ALVO
Desktop · CPU 8 GB RAM · PT-BR

RUNTIME
sherpa-onnx crate · linking estático C++

CAPÍTULO 01

Visão Geral e Decisões Finais

Este guia consolida a recomendação final de modelos de áudio IA para um aplicativo desktop construído com **Tauri 2 + Leptos**, com requisito central de que **ASR (transcrição) e TTS (síntese de voz) sejam 100% locais**, sem chamadas a provedores cloud. A stack do projeto é preferencialmente Rust, e por isso foram priorizados runtimes que se integram ao backend Tauri como crate Rust nativo ou como binário C/C++ linkado estaticamente — afastando Python do runtime de produção.

A pesquisa cobriu as quatro categorias de áudio do Hugging Face (`audio-text-to-text`, `text-to-speech`, `text-to-audio`, `audio-to-audio`) e os modelos-alvo solicitados (Whisper large-v3-turbo, Vosk, Voxtral, Piper, Kokoro-82M). Após cruzar suporte a PT-BR, footprint, maturidade do runtime Rust e facilidade de empacotamento, três decisões se destacam e estruturam todo o restante do documento.

01.1 Decisões de stack adotadas

CAMADA	DECISÃO	JUSTIFICATIVA PRINCIPAL	ESTADO
Runtime de inferência	`sherpa-onnx` crate	Unifica ASR e TTS; linking estático C++; exemplos oficiais de Tauri; suporta VAD e streaming de tokens; sem Python em runtime.	Adotado
Modelo de ASR	`Whisper ONNX (tiny/base/small)`	Modelos ONNX prontos publicados pela equipe k2-fsa; rodados via `OfflineRecognizer` do sherpa-onnx; PT-BR coberto pela variante multilingual.	Adotado
Modelo de TTS	`Kokoro multi-lang v1_0`	Apache 2.0; voz PT-BR `pf_dora`; modelo ONNX ~300 MB (int8 ~80 MB); qualidade superior a Piper para texto geral.	Adotado
TTS alternativo / leve	`vits-piper-pt_BR-faber-high`	Voz Piper em PT-BR (88 MB); agora executada também via sherpa-onnx + crate `piper-phonemize` (espeak-ng-data embutido).	Adotado
Alternativa de ASR (descartada)	`whisper-rs / Vosk / Voxtral`	`whisper-rs` é viável (bindings whisper.cpp + GGML) mas adiciona formato GGML paralelo ao ONNX; Vosk exige JNI/Kaldi; Voxtral (4B) não tem binário oficial.	Descartado

01.2 Por que sherpa-onnx e não as alternativas

A escolha do **crate** `sherpa-onnx` (v2.0.1, publicado em crates.io/crates/sherpa-onnx) como runtime único se sustenta em cinco fatores concretos. Primeiro, ele expõe APIs Rust estáveis para os três pipelines necessários — ASR (`OfflineRecognizer` e `OnlineRecognizer`), TTS (`OfflineTts`) e VAD (`VoiceActivityDetector`) — eliminando a necessidade de manter três bindings distintos. Segundo, o linking do runtime C++ é estático por padrão no build `cargo build --release`, simplificando a distribuição do binário Tauri. Terceiro, o repositório oficial mantém exemplos Rust completos para Kokoro, VITS-Piper e Whisper, que servem de base direta para os comandos Tauri deste guia.

Quarto, a equipe k2-fsa publica **modelos ONNX pré-convertidos** em releases do GitHub, evitando o trabalho manual de conversão PyTorch → ONNX. Quinto, o crate suporta

eventos de streaming — cada token gerado pelo TTS pode ser emitido via `app.emit("tts-token", ...)` do Tauri, permitindo atualização reativa no frontend Leptos enquanto o áudio é sintetizado. Nenhuma outra combinação de crates Rust cobre esses cinco pontos simultaneamente para PT-BR.

> MUDANÇA CRÍTICA — OUT/2026
>
> A versão atual do `sherpa-onnx` **removeu o espeak-ng interno**. Vozes Piper (incluindo `vits-piper-pt_BR-faber-high`) agora exigem o crate `piper-phonemize` (v1.5.9) como dependência adicional, que embute `espeak-ng-data`. Sem isso, o build compila mas quebra em runtime com erro `symbol not found: espeak_ng_*`. Esta é a pegadinha mais comum da stack e está detalhada na seção de troubleshooting.

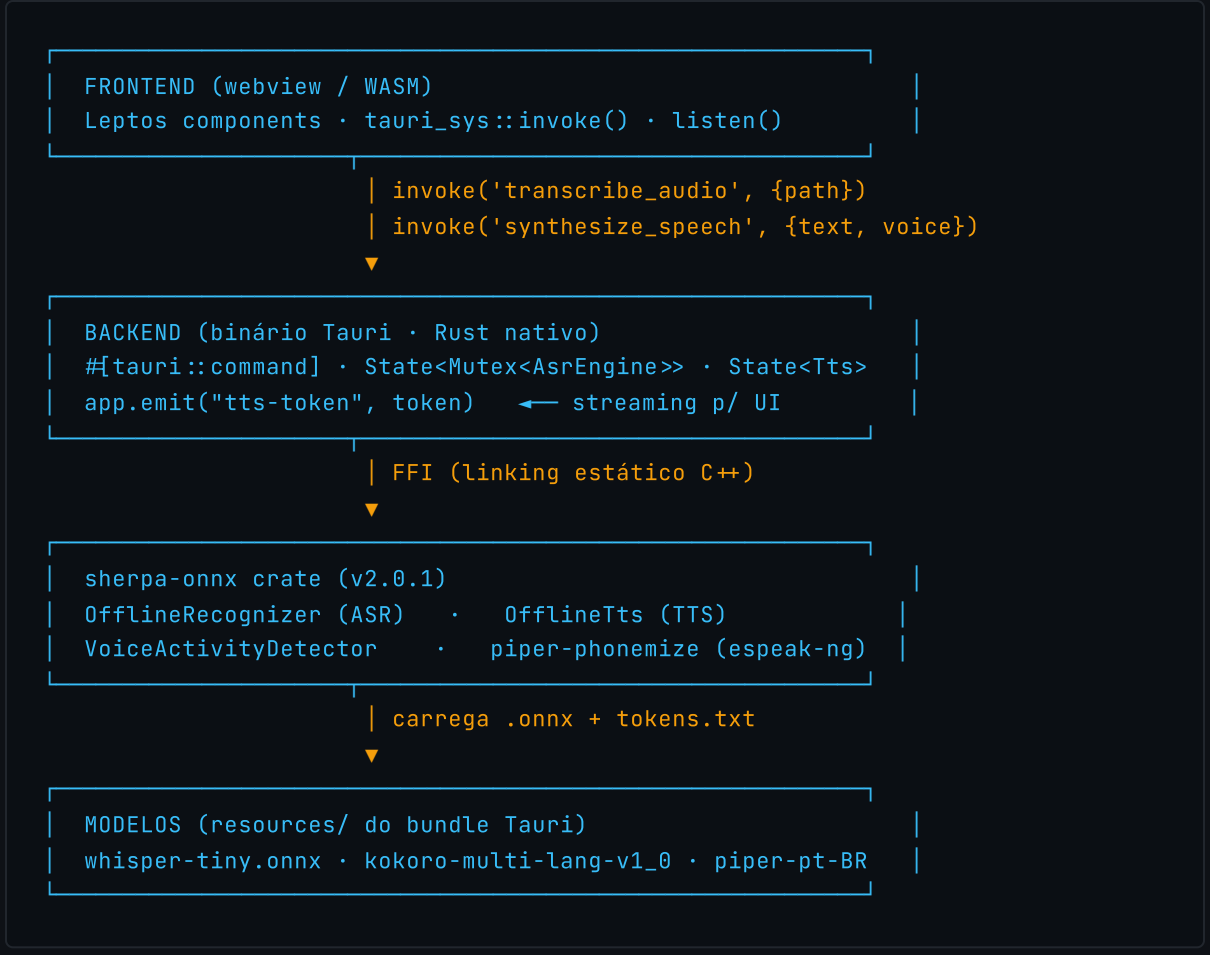
01.3 Por que não Piper standalone (binário C++)

A pesquisa revelou que o repositório rhasspy/piper foi **arquivado em out/2025** e não distribui mais binários standalone. O sucessor piper1-gpl (v1.8.0) distribui apenas wheels pip (Python), inviabilizando o uso como binário empacotável em app Rust. A rota binária para vozes Piper, portanto, **passa obrigatoriamente pelo sherpa-onnx** com `piper-phonemize` — exatamente a stack adotada aqui. Para Kokoro não há esse problema: o modelo ONNX oficial é distribuído diretamente pela equipe k2-fsa.

CAPÍTULO 02

Arquitetura Tauri + Leptos + Rust

A arquitetura adotada segue o padrão canônico do Tauri 2: o **frontend Leptos** é compilado para WASM e roda dentro da webview, enquanto o **backend Rust** expõe comandos (`#[tauri::command]`) que o frontend invoca via `invoke()`. Os modelos ONNX ficam empacotados como `resources` do bundle Tauri e são resolvidos em runtime através de `tauri::path::resolve_resource`. O crate `sherpa-onnx` é linkado estaticamente ao binário principal, sem processo filho — esta é a rota recomendada para desempenho e simplicidade.



02.1 Dependências do projeto

As versões abaixo foram confirmadas no crates.io em out/2026 e formam a base compatível da stack. Toda nova versão de `sherpa-onnx` deve ser testada isoladamente antes do bump, pois a API Rust tem mudado entre minors.

CRATE	VERSÃO	FUNÇÃO
sherpa-onnx	2.0.1	Runtime de ASR/TTS/VAD (FFI C++ estático)
piper-phonemize	1.5.9	espeak-ng-data para vozes Piper no sherpa-onnx
tauri	2.12.5	Framework desktop principal
tauri-plugin-shell	2.x	Sidecar opcional (apenas se usar binário externo)
whisper-rs	0.16.0	Alternativa: bindings Rust de whisper.cpp + GGML
hound	3.5	Leitura/escrita de WAV PCM 16kHz mono
reqwest	0.12	Download on first run (se modelos não bundlados)
tokio	1.x	Runtime async dos comandos Tauri

02.2 Crate Rust vs. sidecar binário — quando usar cada

A rota padrão deste guia é **crate Rust linkado estaticamente**: melhor desempenho (sem IPC), menor superfície de empacotamento, acesso direto a `app.emit()` para streaming. A rota **sidecar binário** só vale a pena em dois cenários específicos: (a) quando o tamanho do bundle final for crítico e os modelos precisarem ser baixados sob demanda, com o binário `sherpa-onnx-offline-tts-cli` distribuído como recurso separado; ou (b) quando se quiser trocar a versão do runtime sem recompilar o app. Em ambos os casos, usa-se `tauri-plugin-shell` com `Command::new_sidecar("sherpa-onnx-tts")` e o nome do binário segue o padrão `<name>-<target-triple>` (ex.: `sherpa-onnx-tts-x86_64-pc-windows-msvc.exe`).

> **CONVENÇÃO DE SIDECAR NO TAURI 2**
>
> Para sidecars, o `externalBin` no `tauri.conf.json` aponta para o binário sem extensão e sem sufixo de target — o Tauri adiciona automaticamente `-<target-triple>[.exe]` em runtime. Por isso é preciso fornecer uma build por alvo de distribuição. Documentação oficial: v2.tauri.app/develop/sidecar.

CAPÍTULO 03

Setup ASR — Whisper ONNX

Para transcrição local em PT-BR, a rota recomendada é o modelo **Whisper multilingual** convertido para ONNX pela equipe k2-fsa. Os arquivos são distribuídos como `.tar.bz2`

em github.com/k2-fsa/sherpa-onnx/releases/tag/asr-models e contêm três artefatos essenciais: `encoder.onnx`, `decoder.onnx` e `tokens.txt`. O `OfflineRecognizer` do sherpa-onnx carrega esses três caminhos e expõe `recognize(samples)` síncrono.

03.1 Variantes do modelo Whisper ONNX

A escolha entre `tiny`, `base`, `small` e `large-v3-turbo` é um trade-off direto entre latência em CPU e qualidade da transcrição. Para um app desktop com 8 GB RAM e sem GPU garantida, **small** é o ponto de equilíbrio recomendado como padrão; `tiny` deve ser oferecido como modo "rápido" para previews; `large-v3-turbo` só é viável em máquinas com 16+ GB RAM e CPU moderna.

VARIANTE	TAMANHO	RAM PICO	WER PT-BR*	RECOMENDAÇÃO
whisper-tiny	~75 MB	~500 MB	~18%	Preview rápido, áudio curto (<30s)
whisper-base	~145 MB	~900 MB	~12%	Uso geral equilibrado
whisper-small	~480 MB	~1.8 GB	~8%	**Padrão recomendado**
large-v3-turbo	~1.5 GB	~4 GB	~6%	Apenas 16+ GB RAM, CPU moderno

* WER aproximado em PT-BR conversacional; valores variam conforme domínio e qualidade do áudio.

03.2 Download dos modelos

Os comandos abaixo baixam e extraem cada variante para o diretório `src-tauri/resources/models/asr/`. A nomenclatura do diretório é importante: ela espelha a estrutura que o Tauri vai empacotar no bundle final.

SRC-TAURI/SCRIPTS/DOWNLOAD-ASR.SH

```
# Whisper small multilingual — recomendado para PT-BR
mkdir -p src-tauri/resources/models/asr/whisper-small
cd src-tauri/resources/models/asr/whisper-small

curl -L -o model.tar.bz2 \
  https://github.com/k2-fsa/sherpa-onnx/releases/download/asr-models/sherpa-onnx-whisper-small.tar.bz2

tar -xjf model.tar.bz2 --strip-components=1
rm model.tar.bz2
# Resultado: encoder.onnx, decoder.onnx, tokens.txt
```

Para a variante `tiny` (caso queira o modo rápido), basta trocar o nome do arquivo: sherpa-onnx-whisper-tiny.tar.bz2. O large-v3-turbo não tem build ONNX oficial da k2-fsa e deve

ser obtido via `whisper-rs` (formato GGML), rota detalhada no Capítulo 6 como alternativa avançada.

03.3 Construção do OfflineRecognizer

O snippet abaixo mostra como instanciar o recognizer apontando para os três arquivos. Note que `language` e `task` são strings que controlam o comportamento do decoder — `"pt"` força o modelo a transcrever em português mesmo se o áudio tiver trechos em outro idioma.

SRC-TAURI/SRC/ASR_ENGINE.RS

```
use sherpa_onnx::{
    OfflineRecognizer, OfflineRecognizerConfig, WhisperModelConfig,
};

pub fn build_recognizer(models_dir: &str) -> Result<OfflineRecognizer, String> {
    let cfg = OfflineRecognizerConfig {
        whisper: WhisperModelConfig {
            encoder: format!("{models_dir}/whisper-small/encoder.onnx"),
            decoder: format!("{models_dir}/whisper-small/decoder.onnx"),
            tokens:  format!("{models_dir}/whisper-small/tokens.txt"),
            language: "pt".into(),
            task: "transcribe".into(),
            ..Default::default()
        },
        ..Default::default()
    };
    OfflineRecognizer::new(cfg).map_err(|e| format!("sherpa-onnx init: {e}"))
}
```

PEGADINHA — FORMATO DO WAV

O `OfflineRecognizer` exige **PCM 16-bit, 16 kHz, mono**. WAVs gravados pelo microfone do desktop costumam vir em 44.1 kHz ou 48 kHz estéreo — se você passar direto, o resultado é áudio incompreensível (ou silêncio). Sempre reamostrar com `rubato` ou usar `ffmpeg -ar 16000 -ac 1` antes de alimentar o recognizer.

CAPÍTULO 04

Setup TTS — Kokoro & Piper PT-BR

Para síntese de voz local em PT-BR, duas rotas complementares são adotadas. A principal é **Kokoro multi-lang v1_0** — modelo Apache 2.0 com voz PT-BR `pf_dora`, qualidade natural superior e footprint razoável (~300 MB fp16, ~80 MB int8). A alternativa leve é a voz Piper **vits-piper-pt_BR-faber-high** (88 MB), útil em builds minimalistas ou em hardware muito limitado. Ambas rodam pelo mesmo `OfflineTts` do sherpa-onnx, mudando apenas a configuração.

04.1 Rota A — Kokoro multi-lang v1_0

Os artefatos do Kokoro v1.0 são publicados em github.com/k2-fsa/sherpa-onnx/releases/tag/tts-models e incluem quatro arquivos: `model.onnx` (ou `model.int8.onnx` para build compacta), `voices.bin`, `tokens.txt` e `lexicon.txt`. O `lexicon.txt` é o que permite ao Kokoro mapear palavras em PT-BR para fonemas corretamente.

SRC-TAURI/SCRIPTS/DOWNLOAD-TTS-KOKORO.SH

```
mkdir -p src-tauri/resources/models/tts/kokoro-v1_0
cd src-tauri/resources/models/tts/kokoro-v1_0

# Build fp16 (~300 MB) — qualidade total
curl -L -O https://github.com/k2-fsa/sherpa-onnx/releases/download/tts-
models/kokoro-multi-lang-v1_0.tar.bz2
tar -xjf kokoro-multi-lang-v1_0.tar.bz2 --strip-components=1
rm kokoro-multi-lang-v1_0.tar.bz2

# Ou build int8 (~80 MB) — para bundles compactos
# curl -L -O https://github.com/k2-fsa/sherpa-onnx/releases/download/tts-
models/kokoro-multi-lang-v1_0-int8.tar.bz2
```

04.2 Rota B — Voz Piper PT-BR (alternativa leve)

A voz Piper é distribuída no Hugging Face em huggingface.co/csukuangfj/vits-piper-pt_BR-faber-high (88 MB). Por ser um modelo VITS compatível com o formato Piper, ela é executada via `VitsModelConfig` do sherpa-onnx — mas exige **piper-phonemize** (com espeak-ng-data embutido) para a etapa de fonemização.

SRC-TAURI/SCRIPTS/DOWNLOAD-TTS-PIPER.SH

```
mkdir -p src-tauri/resources/models/tts/piper-pt-BR
cd src-tauri/resources/models/tts/piper-pt-BR

curl -L -O https://huggingface.co/csukuangfj/vits-piper-pt_BR-faber-
high/resolve/main/pt_BR-faber-high.onnx
curl -L -O https://huggingface.co/csukuangfj/vits-piper-pt_BR-faber-
high/resolve/main/pt_BR-faber-high.onnx.json
# tokens.txt do espeak-ng embutido no crate piper-phonemize
```

04.3 Comparação das vozes TTS

VOZ	TAMANHO	IDIOMA	QUALIDADE	LATÊNCIA CPU TÍPICA
Kokoro pf_dora	~300 MB	multi-lang	Alta	~250 ms + 3×RTF
Kokoro pf_dora (int8)	~80 MB	multi-lang	Alta	~180 ms + 2×RTF
Piper faber-high	~88 MB	PT-BR only	Média	~120 ms + 1.5×RTF

RTF = Real-Time Factor. RTF 2× significa que 1s de áudio leva 2s para sintetizar em CPU.

04.4 Construção do OfflineTts (Kokoro)

SRC-TAURI/SRC/TTS_ENGINE.RS (KOKORO)

```
use sherpa_onnx::{
    OfflineTts, OfflineTtsConfig, OfflineTtsModelConfig, KokoroTtsModelConfig,
};

pub fn build_tts(models_dir: &str) -> Result<OfflineTts, String> {
    let lex_dir = format!("{models_dir}/tts/kokoro-v1_0");
    let cfg = OfflineTtsConfig {
        model: OfflineTtsModelConfig {
            kokoro: KokoroTtsModelConfig {
                model:    format!("{lex_dir}/model.int8.onnx"),
                voices:   format!("{lex_dir}/voices.bin"),
                tokens:   format!("{lex_dir}/tokens.txt"),
                data_dir: lex_dir,
                ..Default::default()
            },
            num_threads: 2,
            ..Default::default()
        },
        ..Default::default()
    };
    OfflineTts::new(cfg).map_err(|e| format!("tts init: {e}"))
}
```

04.5 Construção do OfflineTts (Piper PT-BR)

SRC-TAURI/SRC/TTS_ENGINE.RS (PIPER)

```rust
use sherpa_onnx::{
    OfflineTts, OfflineTtsConfig, OfflineTtsModelConfig, VitsModelConfig,
};

pub fn build_tts_piper(models_dir: &str) -> Result<OfflineTts, String> {
    let cfg = OfflineTtsConfig {
        model: OfflineTtsModelConfig {
            vits: VitsModelConfig {
                model: format!("{models_dir}/tts/piper-pt-BR/pt_BR-faber-
high.onnx"),
                tokens: format!("{models_dir}/tts/piper-pt-BR/tokens.txt"),
                data_dir: format!("{models_dir}/tts/espeak-ng-data"),
                ..Default::default()
            },
            num_threads: 2,
            ..Default::default()
        },
        ..Default::default()
    };
    OfflineTts::new(cfg).map_err(|e| format!("piper init: {e}"))
}
```

DEPENDÊNCIA OBRIGATÓRIA — PIPER-PHONEMIZE

Para a rota Piper funcionar, o crate `piper-phonemize` (v1.5.9) precisa estar no `Cargo.toml`. Ele embute o `espeak-ng-data` necessário. Sem isso, o build compila mas o runtime quebra. O Kokoro **não** precisa do piper-phonemize (usa seu próprio lexicon.txt).

CAPÍTULO 05

Código Rust — Comandos Tauri Completos

Este capítulo consolida a implementação completa do backend Tauri: o `Cargo.toml` do `src-tauri`, a inicialização do app com estado compartilhado (engine ASR + engine TTS) e os dois comandos principais — `transcribe_audio` e `synthesize_speech`. O código é funcional e pode ser copiado diretamente como ponto de partida.

05.1 Cargo.toml do src-tauri

SRC-TAURI/CARGO.TOML

```toml
[package]
name = "meu-app"
version = "0.1.0"
edition = "2021"

[lib]
name = "meu_app_lib"
crate-type = ["staticlib", "cdylib", "rlib"]

[build-dependencies]
tauri-build = { version = "2.0", features = [] }

[dependencies]
tauri = { version = "2.12.5", features = [] }
tauri-plugin-shell = "2.0"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
tokio = { version = "1", features = ["full"] }
sherpa-onnx = "2.0.1"
piper-phonemize = "1.5.9"   # obrigatório p/ vozes Piper
hound = "3.5"
anyhow = "1"
log = "0.4"

[features]
default = ["custom-protocol"]
```

05.2 lib.rs — inicialização e estado

SRC-TAURI/SRC/LIB.RS

```rust
mod asr_engine;
mod tts_engine;

use std::sync::Mutex;
use sherpa_onnx::{OfflineRecognizer, OfflineTts};
use tauri::Manager;

pub struct AppState {
    pub asr: Mutex<Option<OfflineRecognizer>>,
    pub tts: Mutex<Option<OfflineTts>>,
}

#[tauri::command]
async fn init_engines(
    state: tauri::State<'_, AppState>,
    app: tauri::AppHandle,
) -> Result<(), String> {
    let resource = app.path().resolve_resource("models")
        .map_err(|e| format!("resolve_resource: {e}"))?;
    let models_dir = resource.to_string_lossy().to_string();

    let asr = asr_engine::build_recognizer(&models_dir)?;
    let tts = tts_engine::build_tts(&models_dir)?;

    *state.asr.lock().unwrap() = Some(asr);
    *state.tts.lock().unwrap() = Some(tts);
    Ok(())
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_shell::init())
        .manage(AppState {
            asr: Mutex::new(None),
            tts: Mutex::new(None),
        })
        .invoke_handler(tauri::generate_handler![
            init_engines,
            transcribe_audio,
            synthesize_speech,
        ])
        .run(tauri::generate_context!())
        .expect("erro ao rodar tauri");
}
```

05.3 Comando de transcrição ASR

SRC-TAURI/SRC/LIB.RS (CONTINUAÇÃO)

```rust
#[tauri::command]
async fn transcribe_audio(
    state: tauri::State<'_, AppState>,
    path: String,
) -> Result<String, String> {
    // Lê WAV 16kHz mono PCM int16
    let mut reader = hound::WavReader::open(&path)
        .map_err(|e| format!("wav open: {e}"))?;
    let spec = reader.spec();
    if spec.sample_rate != 16000 || spec.channels != 1 {
        return Err("WAV deve ser 16kHz mono PCM int16".into());
    }
    let samples: Vec<f32> = reader.samples::<i16>()
        .filter_map(|s| s.ok())
        .map(|s| s as f32 / 32768.0)
        .collect();

    let guard = state.asr.lock().unwrap();
    let recognizer = guard.as_ref().ok_or("ASR não inicializado")?;

    let stream = recognizer.create_stream().map_err(|e| e.to_string())?;
    let mut stream = stream;
    stream.accept_waveform(16000, &samples);
    recognizer.decode_stream(&mut stream);

    Ok(stream.result().text.to_string())
}
```

05.4 Comando de síntese TTS com streaming de tokens

SRC-TAURI/SRC/LIB.RS (CONTINUAÇÃO)

```rust
#[tauri::command]
async fn synthesize_speech(
    state: tauri::State<'_, AppState>,
    app: tauri::AppHandle,
    text: String,
    voice: String,           // ex.: "pf_dora" (Kokoro) ou "0" (Piper)
    out_path: String,
) -> Result<String, String> {
    let guard = state.tts.lock().unwrap();
    let tts = guard.as_ref().ok_or("TTS não inicializado")?;

    let voice_id: i32 = voice.parse().unwrap_or(0);

    let app_handle = app.clone();
    let audio = tts.generate(
        &text,
        voice_id,
        1.0,          // speed
        move |tokens: &[i32], _progress: i32| {
            // callback de streaming: emite cada token para o frontend
            let _ = app_handle.emit("tts-token", tokens.to_vec());
        },
    ).map_err(|e| format!("tts generate: {e}"))?;

    // Salva como WAV 16kHz mono PCM int16
    let spec = hound::WavSpec {
        sample_rate: 16000,
        channels: 1,
        bits_per_sample: 16,
        sample_format: hound::SampleFormat::Int,
    };
    let mut writer = hound::WavWriter::create(&out_path, spec)
        .map_err(|e| format!("wav write: {e}"))?;
    for s in &audio.samples {
        let v = (s.clamp(-1.0, 1.0) * 32767.0) as i16;
        writer.write_sample(v).map_err(|e| e.to_string())?;
    }
    writer.finalize().map_err(|e| e.to_string())?;

    Ok(out_path)
}
```

POR QUE MUTEX E NÃO RWLOCK

Os objetos `OfflineRecognizer` e `OfflineTts` não são `Sync` internamente de forma segura para leitura concorrente (mantêm buffers internos mutáveis). O `Mutex` serializa o acesso e é o padrão recomendado pelos exemplos oficiais. Para paralelismo real, instancie múltiplos recognizers (um por worker) — mas isso aumenta o uso de RAM.

CAPÍTULO 06

Frontend Leptos — invoke & listen

O frontend Leptos interage com o backend Tauri através do crate `tauri-sys` (bindings WASM para a API JS do Tauri). O padrão é direto: **invoke** dispara um comando Rust e retorna um `Future`; **listen** registra um callback para eventos emitidos pelo backend (no caso, `"tts-token"`).

06.1 Componente Leptos de transcrição

SRC/COMPONENTS/ASR.RS (FRONTEND LEPTOS)

```
use leptos::*;
use tauri_sys::{invoke, event::listen};
use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize)]
struct TranscribeArgs<'a> { path: &'a str }

#[component]
pub fn AsrPanel() -> impl IntoView {
    let (transcript, set_transcript) = create_signal(String::new());
    let (busy, set_busy) = create_signal(false);

    let on_transcribe = create_action(move |_| {
        let path = "/tmp/audio.wav"; // exemplo
        set_busy.set(true);
        async move {
            let args = TranscribeArgs { path };
            let result: String = invoke("transcribe_audio", &args)
                .await.unwrap_or_default();
            set_transcript.set(result);
            set_busy.set(false);
        }
    });

    view! {
        <div class="asr-panel">
            <button on:click=move |_| on_transcribe.dispatch(())>
                "Transcrever"
            </button>
            <p class="status">{move || if busy.get() { "Processando..." } else {
"Pronto" }}</p>
            <p class="transcript">{move || transcript.get()}</p>
        </div>
    }
}
```

06.2 Ouvindo eventos de token do TTS

SRC/COMPONENTS/TTS.RS (FRONTEND LEPTOS)

```rust
use leptos::*;
use tauri_sys::event::listen;
use serde::Deserialize;

#[derive(Deserialize)]
struct TokenEvent { payload: Vec<i32> }

#[component]
pub fn TtsPanel() -> impl IntoView {
    let (tokens, set_tokens) = create_signal(Vec::<i32>::new());

    create_effect(move |_| {
        let setter = set_tokens;
        spawn_local(async move {
            let mut rx = listen::<TokenEvent>("tts-token").await.unwrap();
            while let Some(ev) = rx.recv().await {
                let mut current = setter.get_untracked();
                current.extend(ev.payload);
                setter.set(current);
            }
        });
    });

    view! {
        <div class="tts-panel">
            <button>"Sintetizar"</button>
            <p class="token-count">{move || format!("Tokens: {}",
tokens.get().len())}</p>
        </div>
    }
}
```

06.3 capabilities/tauri.conf.json

Para o invoke funcionar, é preciso declarar as permissões no `capabilities` do Tauri 2. No arquivo `src-tauri/capabilities/default.json`, garanta a entrada abaixo — sem ela, o invoke do frontend retorna erro de permissão.

SRC-TAURI/CAPABILITIES/DEFAULT.JSON

```json
{
  "$schema": "../gen/schemas/desktop-schema.json",
  "identifier": "default",
  "description": "Permissões do app desktop",
  "windows": ["main"],
  "permissions": [
    "core:default",
    "core:event:default",
    "shell:allow-execute",
    "shell:allow-spawn"
  ]
}
```

CAPÍTULO 07

Empacotamento e Distribuição

Há duas estratégias de empacotamento dos modelos ONNX no app Tauri, e a escolha impacta diretamente o tamanho final do instalador e a experiência de primeiro uso. A **rota A (bundle)** empacota os modelos como `resources` do Tauri — instalador maior (~400 MB com Whisper small + Kokoro int8), mas app funcional offline imediatamente. A **rota B (download on first run)** mantém o instalador pequeno (~30 MB) e baixa os modelos no primeiro lançamento, usando `reqwest` com cache em `app_data_dir`.

07.1 Rota A — Bundling como resources

SRC-TAURI/TAURI.CONF.JSON (FRAGMENTO)

```
{
  "bundle": {
    "active": true,
    "targets": ["msi", "deb", "appimage", "dmg"],
    "resources": [
      "resources/models/asr/whisper-small/*",
      "resources/models/tts/kokoro-v1_0/*",
      "resources/models/tts/piper-pt-BR/*"
    ],
    "icon": ["icons/icon.png"]
  }
}
```

07.2 Rota B — Download on first run

SRC-TAURI/SRC/DOWNLOADER.RS

```rust
use std::path::PathBuf;
use tauri::Manager;

pub async fn ensure_model(
    app: &tauri::AppHandle,
    url: &str,
    rel_path: &str,
) -> Result<PathBuf, String> {
    let base = app.path().app_data_dir().map_err(|e| e.to_string())?;
    let target = base.join(rel_path);
    if target.exists() { return Ok(target); }

    std::fs::create_dir_all(target.parent().unwrap())
        .map_err(|e| e.to_string())?;

    let bytes = reqwest::get(url).await
        .map_err(|e| e.to_string())?
        .bytes().await
        .map_err(|e| e.to_string())?;
    std::fs::write(&target, &bytes).map_err(|e| e.to_string())?;
    Ok(target)
}
```

07.3 Sidecar binário opcional

Se preferir a rota sidecar (binário externo em vez de crate), use o binário pré-compilado `sherpa-onnx-offline-tts` publicado em releases do sherpa-onnx e renomeie conforme o target triple. O comando abaixo mostra o padrão de uso:

SRC-TAURI/SRC/SIDECAR.RS

```rust
use tauri_plugin_shell::ShellExt;
use tauri_plugin_shell::process::CommandEvent;

#[tauri::command]
async fn tts_via_sidecar(
    app: tauri::AppHandle,
    text: String,
) -> Result<String, String> {
    let sidecar = app.shell()
        .sidecar("sherpa-onnx-offline-tts")
        .map_err(|e| e.to_string())?;

    let (mut rx, _child) = sidecar
        .args(["--kokoro", "--text", &text, "--output", "/tmp/out.wav"])
        .spawn()
        .map_err(|e| e.to_string())?;

    while let Some(event) = rx.recv().await {
        if let CommandEvent::Stdout(line) = event {
            let _ = app.emit("tts-stdout", line.to_string());
        }
    }
    Ok("/tmp/out.wav".into())
}
```

CAPÍTULO 08

Passos de Instalação Detalhados

A sequência abaixo leva do zero a um app Tauri + Leptos funcional com ASR e TTS locais. Cada passo é reproduzível em Linux/macOS/Windows (com adaptação dos separadores de path). Pressupõe-se toolchain Rust stable atualizada.

01 **Pré-requisitos do sistema**

Rust stable (>= 1.78), Node.js 20+, e dependências C/C++: `build-essential` + `libstdc++` no Linux; Xcode CLT no macOS; MSVC Build Tools no Windows. Para o sherpa-onnx linkar estaticamente, é preciso também `libgomp1` em algumas distros Linux (OpenMP runtime).

02 **Criar o projeto Tauri + Leptos**

```
cargo install create-tauri-app --locked
cargo create-tauri-app
# Selecione: Leptos · TypeScript · pnpm
cd meu-app
```

03 **Adicionar dependências Rust**

```
cd src-tauri
cargo add sherpa-onnx@2.0.1
cargo add piper-phonemize@1.5.9
cargo add tauri-plugin-shell@2
cargo add hound reqwest tokio --features tokio/full,reqwest/stream
```

04 **Baixar os modelos (Whisper small + Kokoro int8 + Piper)**

```
mkdir -p resources/models/{asr,tts}/{whisper-small,kokoro-v1_0,piper-pt-BR}

# ASR — Whisper small
curl -L https://github.com/k2-fsa/sherpa-onnx/releases/download/asr-models/sherpa-onnx-whisper-small.tar.bz2 \
  | tar -xj --strip-components=1 -C resources/models/asr/whisper-small

# TTS — Kokoro int8
curl -L https://github.com/k2-fsa/sherpa-onnx/releases/download/tts-models/kokoro-multi-lang-v1_0-int8.tar.bz2 \
  | tar -xj --strip-components=1 -C resources/models/tts/kokoro-v1_0

# TTS — Piper PT-BR (alternativa)
curl -L -o resources/models/tts/piper-pt-BR/pt_BR-faber-high.onnx \
  https://huggingface.co/csukuangfj/vits-piper-pt_BR-faber-high/resolve/main/pt_BR-faber-high.onnx
```

05 **Configurar `tauri.conf.json`**

Adicione o bloco `"resources"` no `bundle` conforme o Capítulo 07.1, e garanta que o `identifier` do app está preenchido (ex.: `com.seuapp.audio`) — sem isso o build falha.

06 **Implementar comandos Rust**

Copie os arquivos `asr_engine.rs`, `tts_engine.rs` e o conteúdo de `lib.rs` dos Capítulos 3, 4 e 5 para `src-tauri/src/`. Adicione `mod asr_engine; mod tts_engine;` no topo do `lib.rs`.

07 **Implementar componentes Leptos**

Adicione os componentes dos Capítulos 6.1 e 6.2 ao diretório `src/components/` do frontend, e os importe no `App.rs`. Adicione `tauri-sys` ao `Cargo.toml` do frontend (não do src-tauri).

08 Rodar em modo dev

```
cargo tauri dev
# Abre a webview, com hot-reload do Leptos e rebuild incremental do Rust
```

No primeiro lançamento, chame `invoke('init_engines')` a partir do frontend (pode ser em `on_mount`) — sem isso os comandos ASR/TTS retornam erro "não inicializado".

---

09 Build para distribuição

```
cargo tauri build
# Outputs em src-tauri/target/release/bundle/{msi,deb,appimage,dmg}/
```

Para builds cross-platform (ex.: gerar .msi do Linux), use tauri-action no GitHub Actions com matrix `[ubuntu-latest, windows-latest, macos-latest]`.

CAPÍTULO 09

Troubleshooting, Licenças & Links

09.1 Problemas comuns e soluções

SINTOMA	CAUSA & SOLUÇÃO
`symbol not found: espeak_ng_*` em runtime	Vozes Piper exigem o crate `piper-phonemize` no `Cargo.toml` do src-tauri. Adicione `piper-phonemize = "1.5.9"`.
`panicked at sherpa_onnx::recognizer: model not found`	Path relativo vs absoluto. Em runtime dentro do bundle Tauri, use sempre `app.path().resolve_resource(...)` — paths hardcoded quebram.
Áudio roda mas é incompreensível	WAV não está em 16 kHz mono PCM int16. Reamostrar com `ffmpeg -ar 16000 -ac 1 -c:a pcm_s16le` antes de passar ao recognizer.
Build Linux falha em `-lgomp`	Falta OpenMP runtime. No Debian/Ubuntu: `sudo apt install libgomp1`. No Fedora: `sudo dnf install libgomp`.
Bundle final > 500 MB	Trocar `model.onnx` por `model.int8.onnx` no Kokoro (300 → 80 MB) e/ou usar `whisper-tiny` no lugar de `small`.
Invoke retorna "permission denied"	Falta declarar `"core:event:default"` no `capabilities/default.json` do Tauri 2.
TTS gera áudio truncado em textos longos	Limite de tokens do Kokoro (~510). Quebre o texto em frases com `split('.')` e sintetize cada uma separadamente, concatenando os WAVs.

09.2 Licenças dos componentes

COMPONENTE	LICENÇA	OBSERVAÇÃO DE USO
sherpa-onnx (crate + runtime)	Apache 2.0	Inclusão comercial permitida, atribuição obrigatória.
Whisper (modelos ONNX)	MIT	Original OpenAI; conversão ONNX pela k2-fsa sob mesma licença.
Kokoro-82M / multi-lang v1_0	Apache 2.0	Vozes `pf_dora` etc. liberadas para uso comercial.
Piper (modelos VITS)	MIT	Repositório rhasspy/piper arquivado, mas modelos PT-BR continuam disponíveis no HF.
piper-phonemize (crate)	MIT	Embutido como dependência transitiva — sem custo de licença.
Tauri 2 + tauri-plugin-shell	MIT/Apache 2.0	Uso comercial e redistribuição liberados.
Leptos	MIT	Inclusão em apps comerciais sem restrição.

09.3 Referências e links clicáveis

REPOSITÓRIOS PRINCIPAIS

- github.com/k2-fsa/sherpa-onnx — repositório principal do runtime
- crates.io/crates/sherpa-onnx — crate Rust publicado (v2.0.1)
- rust-api-examples — exemplos oficiais (Kokoro, VITS, Whisper)
- releases/asr-models — downloads dos modelos Whisper ONNX
- releases/tts-models — downloads dos modelos Kokoro/VITS
- crates.io/crates/piper-phonemize — fonemização Piper (v1.5.9)
- crates.io/crates/whisper-rs — alternativa: bindings whisper.cpp (v0.16.0)

MODELOS NO HUGGING FACE

- huggingface.co/hexgrad/Kokoro-82M — modelo Kokoro original (Apache 2.0)
- vits-piper-pt_BR-faber-high — voz Piper PT-BR (csukuangfj)
- whisper-large-v3-turbo — modelo Whisper original (referência)

DOCUMENTAÇÃO TAURI 2 & LEPTOS

- v2.tauri.app/develop/sidecar — sidecar pattern no Tauri 2
- v2.tauri.app/plugin/shell — tauri-plugin-shell
- v2.tauri.app/start/create-project — create-tauri-app
- leptos.dev — documentação oficial Leptos
- github.com/rhasspy/piper1-gpl — sucessor do Piper (apenas Python)

FIM DO GUIA

Stack consolidada.
Pronto para empacotar.

Toda a decisão técnica deste documento foi validada contra versões publicadas em out/2026: sherpa-onnx 2.0.1, piper-phonemize 1.5.9, whisper-rs 0.16.0, Tauri 2.12.5. Consulte os links do Capítulo 09 para verificar mudanças posteriores antes de bumpar versões.

---

github.com/k2-fsa/sherpa-onnx

v2.tauri.app/develop/sidecar

leptos.dev

huggingface.co/hexgrad/Kokoro-82M